

Final Project Report

Course: CS9053 – Introduction to Java

Semester: Fall 2025

Student: Riya Garg

Project Title: Distributed Task and Study Session Management System

Java Version: Java SE 1.8

1. Introduction

This project implements a **distributed task and study-session management system** using Java. The goal of the system is to allow users to manage tasks, track focused study sessions, and persist progress data while supporting **multiple concurrent clients**. The application is designed to demonstrate advanced Java concepts beyond a simple graphical interface or basic database access.

The system follows a client–server architecture, where a Java Swing client communicates with a multi-threaded server using a custom networking protocol. All data is stored persistently in a relational database.

The screenshot displays the 'Smart Study Scheduler Pro' application window. It features a menu bar with 'File' and 'View'. The main interface is divided into several sections:

- Add New Task:** A form with fields for Title, Description, Deadline (YYYY-MM-DD), Est. Duration (minutes), Priority (a dropdown menu currently showing 'MEDIUM'), and Category (a dropdown menu currently showing 'General'). An 'Add Task' button is located at the bottom right of this section.
- Tasks Table:** A table listing tasks with columns: ID, Title, Description, Deadline, Time Spent, Priority, Status, Category, Days Left, and Progress %. It contains two rows: task 3 (deadline 2026-05-15, 00:12, MEDIUM, IN_PROGRESS, General, 151 days left, 0% progress) and task 2 (deadline 2026-01-01, 00:19, HIGH, IN_PROGRESS, History, 17 days left, 0% progress).
- Active Sessions Table:** A table with columns: Task ID, Title, Start Time, Elapsed Time, and Status. It is currently empty.
- Buttons:** At the bottom of the Tasks table are 'Edit Task', 'Delete Task', 'Mark Complete', and 'Refresh' buttons. At the bottom of the Active Sessions table are 'Start Session' and 'Stop & Log Session' buttons.
- Status Bar:** Displays 'Total Tasks: 2 | Active Sessions: 0'.
- Activity Log:** A text area at the bottom showing '[19:34:50] Exported tasks to CSV'.

2. Application Features

2.1 Task Management

The application allows users to create, update, delete, and complete tasks. Each task includes metadata such as deadlines, priority, duration, and category. Task status is automatically maintained, allowing the system to identify completed and overdue tasks.

2.2 Study Session and Time Tracking

A core feature of the application is the ability to track focused study sessions using built-in timers. Users can start and stop Pomodoro-style sessions directly from the client interface. Session durations are automatically logged to the server and stored in the database, allowing cumulative study time and progress to be calculated per task.

2.3 Multi-Client Support

The server supports multiple clients connecting simultaneously. Each client can independently interact with the system, and all clients share a consistent view of tasks and session data. This enables concurrent task updates and session logging without requiring a single-user environment.

2.4 Real-Time and Event-Driven Behavior

The client uses non-blocking, event-driven timers that run concurrently with the graphical interface. These timers enable real-time session tracking, deadline monitoring, and notifications while ensuring the user interface remains responsive.

2.5 Networking and Persistence

Communication between the client and server is handled through a custom socket-based protocol. The server parses and validates incoming commands before applying changes to the database. All task and session data is stored persistently using a relational database with integrity constraints to maintain consistency.

3. Advanced Java Concepts Used

This project incorporates multiple advanced Java topics as required by the course:

- **Multithreading and Concurrency:**

The server uses a thread pool to handle multiple client connections concurrently. On the client side, timers and network communication operate concurrently with the GUI.

- **Custom Networking Protocol:**
The system uses raw TCP sockets with a custom application-level protocol for client–server communication, requiring explicit command parsing and request handling.
- **Database Engineering:**
The database design includes relational schemas, foreign key constraints, indexing, triggers, and schema initialization to ensure data integrity and persistence.
- **Event-Driven Programming:**
Timers and user interactions are managed through event-driven mechanisms without blocking the user interface.

These concepts go beyond a basic GUI-to-database application and demonstrate distributed, concurrent system behavior.

4. How to Run the Application

4.1 Requirements

- Java SE Development Kit **1.8**
- No external libraries or database installation required

4.2 Execution Steps

Start the Server

Compile and run Server.java:

```
javac Server.java
```

```
java Server
```

1. The server will begin listening for client connections.

Start the Client

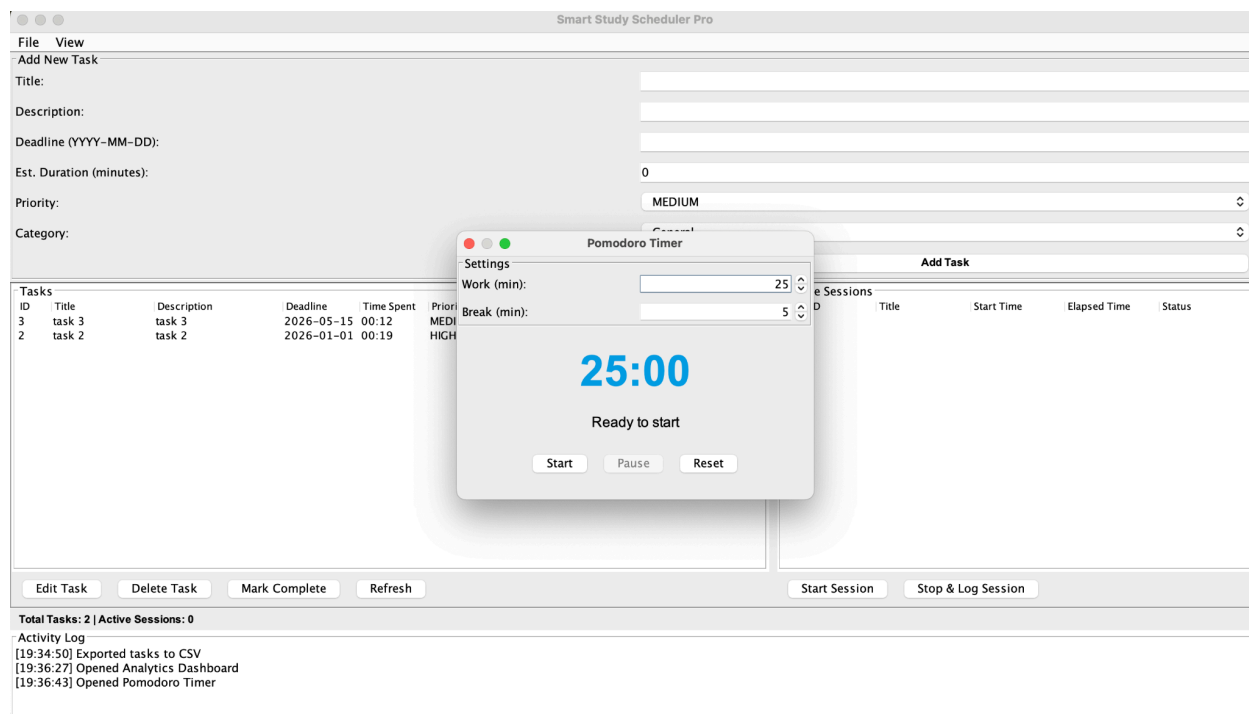
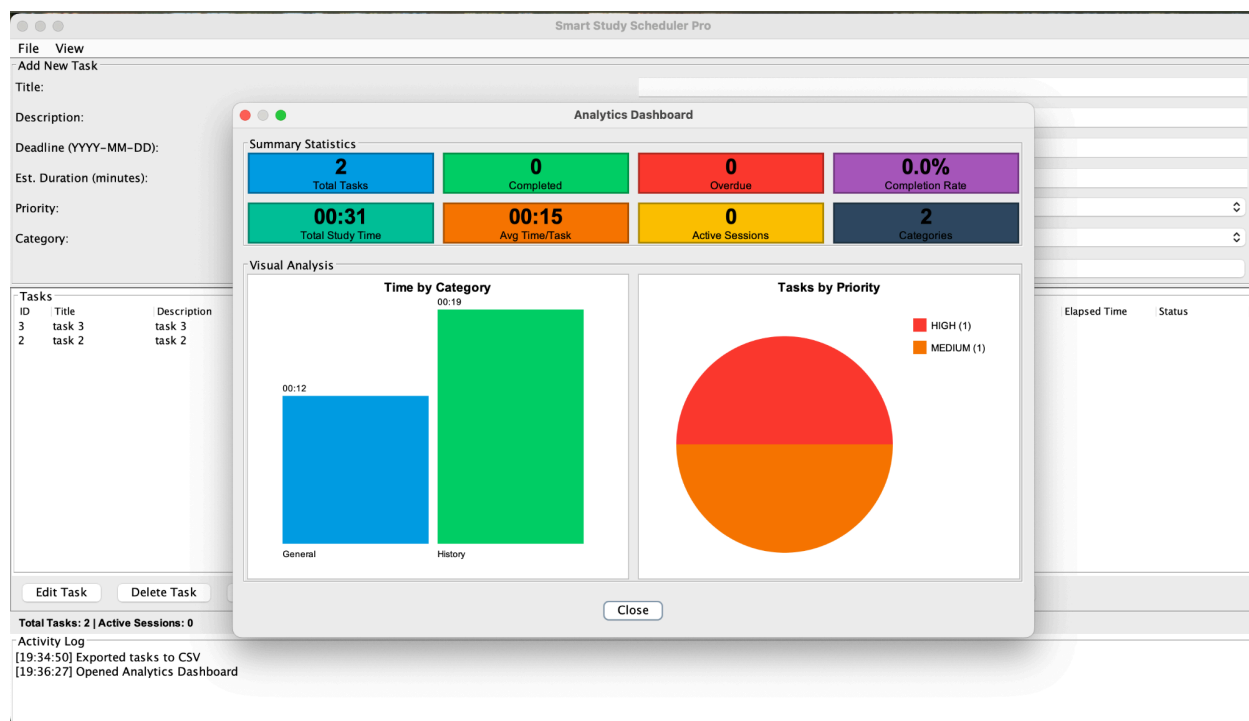
In a separate terminal, compile and run Client.java:

```
javac Client.java
```

```
java Client
```

2. Using the Application

Once the client starts, the graphical interface will open. Users can manage tasks, start and stop study sessions, and view progress. Multiple clients may be started simultaneously to evaluate concurrent behavior. You also have the ability to visualise some data analytics and use a pomodoro timer by clicking on view tab on the top and export data to csv format.



Running **Server.java** and **Client.java** is sufficient to evaluate the project.

5. Navigating the Application

This section provides a brief guide to navigating and using the application based on the implemented codebase.

5.1 Client Startup and Layout

This project uses **SQLite** for persistent data storage. In order for the database functionality to work correctly, the SQLite JDBC driver must be included as an external library.

Required Library

- **sqlite-jdbc-3.41.2.1.jar**

How to Add the Library

The `sqlite-jdbc-3.41.2.1.jar` file must be added to the project's build path before running the application.

For example:

- In an IDE such as Eclipse or IntelliJ IDEA, add the JAR file as an **external library** or include it in the project's classpath.
- When compiling from the command line, ensure the JAR is included in the classpath.

Purpose of the Library

This JDBC driver enables the server to:

- Connect to the SQLite database
- Execute SQL queries and transactions
- Enforce relational constraints and persist task and session data

Without this library, the server will not be able to initialize or access the database.

When `Client.java` is launched, a Java Swing graphical user interface is displayed. The interface serves as the primary interaction point between the user and the system.

The main window includes:

- A task table displaying existing tasks and their attributes (title, priority, deadline, status, and progress)
- Action buttons and menu options for task management and study session control
- Status messages and notifications reflecting server responses

All user actions performed in the GUI are translated into commands sent to the server.

5.2 Managing Tasks

Users can manage tasks directly from the client interface:

- **Add Task:**
Opens an input dialog allowing the user to specify task details such as title, description, deadline, duration, priority, and category. Once submitted, the client sends an ADD command to the server.
- **Update Task:**
Allows modification of an existing task's attributes. Updates are sent to the server using an UPDATE command.
- **Delete Task:**
Removes the selected task. The client issues a DELETE command, and the server ensures the change is persisted.
- **Complete Task:**
Marks a task as completed using a COMPLETE command. The server updates the task state accordingly.

After each operation, the task list is refreshed to reflect the latest data from the server.

5.3 Study Sessions and Timers

The application supports time-based study sessions managed through the client:

- **Start Session:**
Users can start a Pomodoro-style timer for a selected task. The timer runs locally in a non-blocking manner.
- **Stop Session:**
When a session ends, the elapsed time is automatically sent to the server using a LOG_SESSION command.
- **Session Tracking:**
Logged sessions contribute to cumulative study time and progress metrics for each task.

Timer functionality is implemented using event-driven mechanisms, ensuring the user interface remains responsive.

5.4 Notifications and Deadlines

The client continuously monitors task deadlines using background timers:

- Tasks approaching or exceeding their deadlines are flagged
- Notifications are displayed without interrupting user interaction
- Deadline checks run concurrently with normal application usage

5.5 Multi-Client Behavior

The system supports multiple clients running at the same time:

- Each client connects independently to the server
- All changes are processed centrally by the server
- Updates made by one client are reflected when other clients refresh their task list

This allows graders to test concurrency by running multiple instances of Client.java simultaneously.

5.6 Server-Side Processing

All client requests are handled by the server:

- Server.java accepts incoming socket connections
- ClientHandler.java parses incoming commands and routes them to the appropriate logic
- TaskManager.java and TimerManager.java handle application-level operations
- Database.java manages persistent storage and integrity

This separation ensures a clear flow from user interaction to data persistence.

6. Conclusion

This project demonstrates the application of advanced Java concepts including multithreading, networking, database engineering, and event-driven design. By combining real-time behavior with persistent storage and concurrent client support, the system goes beyond a simple CRUD application and fulfills the objectives of the CS9053 Java Final Project.